Section 03 - Architecture Explanation

SalesOps AI is a serverless training and examination platform for sales and service representatives. Managers create customer personas, build exam scenarios, generate realistic inbox issues with AI, and review team performance. Representatives take timed scenario exams, respond to released customer issues, and receive AI-based coaching results.

This section explains the final system architecture in the level of detail expected from the Section 02 architecture diagram. The diagram should show the browser-based React application, Amazon API Gateway, AWS Lambda functions, Amazon Cognito, Amazon DynamoDB tables, Amazon SQS, AWS Secrets Manager, and the external OpenAI Responses API.

1. High-Level Architecture

The system is split into a browser frontend and a serverless AWS backend.

- Frontend: Vite, React, and TypeScript single-page application. The frontend reads the API base URL from `VITE_API_BASE_URL` and calls the deployed AWS REST API with Axios.
- API edge: Amazon API Gateway REST API, deployed as a regional endpoint under the `dev` stage. CORS allows the browser application to call the API.
- Authentication: Amazon Cognito User Pool and web app client. Cognito owns user credentials, confirmation codes, refresh tokens, and password reset.
- Compute: AWS Lambda functions written in Node.js. Each API route maps to a focused Lambda handler. In the AWS Academy lab, all functions use the existing `LabRole` because the lab blocks creating new IAM roles.
- Storage: Amazon DynamoDB tables store application profiles, reusable personas, scenarios, generated issues, exam sessions, issue responses, and evaluations.
- Delayed exam events: Amazon SQS stores delayed issue-release messages. A Lambda consumer marks exam issues visible when their delay expires.
- Secrets: AWS Secrets Manager stores the OpenAI API key in `salesops/dev/llm-api-keys`.
- AI provider: OpenAI Responses API generates scenario issues and evaluates completed exams using strict JSON schemas.

2. API Gateway and Lambda Layer

Amazon API Gateway is the single backend entry point for the frontend. The API is defined in `backend/template.yaml` as an AWS SAM REST API named `SalesOpsApi`. The stage name is configurable and defaults to `dev`.

Public routes are used only for account lifecycle operations:

- `GET /health` verifies that API Gateway can invoke Lambda and that the frontend can reach the backend.
- `POST /auth/signup` creates a Cognito account and an application profile.
- `POST /auth/confirm` confirms the Cognito signup code and activates the profile.
- `POST /auth/resend-confirmation` resends the email confirmation code.
- `POST /auth/signin` authenticates with Cognito and returns tokens plus the app profile.
- `POST /auth/refresh` refreshes Cognito tokens.
- `POST /auth/forgot-password` starts password recovery.
- `POST /auth/forgot-password/confirm` completes password recovery.

Protected routes use the API Gateway Cognito authorizer. The frontend sends `Authorization: Bearer <idToken>`. API Gateway validates the token against the Cognito User Pool before invoking the Lambda. The

Lambda then reads the Cognito `sub` claim and checks the user's application role and status in DynamoDB.

Manager routes include:

- `/users` and `/users/{userId}` for user administration.
- `/dashboard` for aggregated exam analytics.
- `/personas` and `/personas/{personaId}` for customer persona management.
- `/scenarios`, `/scenarios/{scenarioId}`, `/scenarios/{scenarioId}/publish`, `/scenarios/{scenarioId}/clone`, and `/scenarios/{scenarioId}/archive` for scenario lifecycle management.
- `/scenarios/{scenarioId}/issues/generate` and `/scenarios/{scenarioId}/issues/{issueId}` for AI issue generation and editing.

Representative routes include:

- `/exam/scenarios` for listing published, ready-to-use scenarios.
- `/exam/sessions` for creating timed exam sessions.
- `/exam/sessions/{sessionId}/pulse` for reading visible issues and remaining time.
- `/ exam/ sessions/ {sessionId}/ issues/ {issueId}/ responses` for submitting answers.
- `/ exam/ sessions/ {sessionId}/ issues/ {issueId}/ done` for closing an answered issue.
- `/exam/sessions/{sessionId}/evaluation` for creating and reading AI evaluation results.

3. Identity, Roles, and Authorization

Cognito stores credentials. DynamoDB stores application authorization data. This separation keeps passwords and authentication secrets outside the application database.

The `Users` table stores:

- `userId`: Cognito `sub`, used as the partition key.
- `email` and `emailLower`.
- `fullName`.
- `role`: either `rep` or `manager`.
- `status`: `PENDING_CONFIRMATION`, `ACTIVE`, or `SUSPENDED`.
- `createdAt` and `updatedAt`.

The `EmailIndex` global secondary index supports profile lookup by normalized email during signup, signin, and account confirmation.

New users start as representatives with `status=PENDING_CONFIRMATION`. After email confirmation, the profile becomes `ACTIVE`. Managers can promote or suspend users from the Users page, but the backend prevents a manager from demoting or suspending their own manager account.

Every protected business Lambda performs application-level authorization after Cognito authentication:

- `requireManager` checks that the current profile is an active manager.
- `requireRep` checks that the current profile is an active representative.
- Exam ownership checks ensure a representative can only read or modify their own exam session.

4. DynamoDB Data Model

The backend uses four DynamoDB tables, all in pay-per-request mode.

`Users` stores one profile per Cognito user. It contains profile and role data, not passwords.

`Personas` stores reusable customer behavior profiles authored by managers. Each item contains `personaId`, `name`, `description`, `behaviorNotes`, `status`, and timestamps.

`Scenarios` stores manager-authored exams. Each scenario contains `scenarioId`, `title`, `description`, selected `personaIds`, an `issueCount`, generated `issues`, generation metadata, status, and timestamps. Scenario status moves through `DRAFT`, `PUBLISHED`, and `ARCHIVED`.

`ExamSessions` uses a composite key:

- Partition key: `sessionId`.
- Sort key: `recordId`.

This table stores several record types under one session:

- `META`: session metadata, representative owner, scenario reference, duration, start time, end time, and status.
- `ISSUE#<issueId>`: copied issue content, release time, visibility state, answer responses, and done time.
- `EVALUATION`: AI scoring and coaching result for the completed session.

The composite-key design lets the backend read a full exam session with one DynamoDB query on `sessionId`.

5. Manager Content Flow

Managers build the training content library before representatives take exams.

1. A manager signs in through Cognito and receives an ID token.
2. The frontend stores the session in browser local storage and sends the ID token to API Gateway on protected calls.
3. API Gateway validates the token and invokes the relevant content Lambda.
4. The Lambda verifies the user's `manager` role in the `Users` table.
5. The manager creates personas in the `Personas` table.
6. The manager creates scenarios in the `Scenarios` table, selects one or more personas, and chooses the number of issues.
7. The manager publishes the scenario only after at least one persona is selected.
8. The issue generation Lambda reads the scenario and personas, reads the OpenAI key from Secrets Manager, calls the OpenAI Responses API, validates the structured JSON response, and stores editable issues back on the scenario.
9. If OpenAI issue generation fails because of a provider or secret problem, the backend stores clearly marked demo issues so the training flow can continue during the lab.

Generated issues remain editable by managers. This allows a human review step before representatives receive the exam.

6. Representative Exam Flow

Representatives take timed exams based on published scenarios with generated issues.

1. A representative opens `/exam/start`.
2. The frontend calls `/exam/scenarios`.
3. The backend returns only scenarios with `status=PUBLISHED` and at least one generated issue.
4. The representative starts an exam by calling `POST /exam/sessions` with a scenario ID.
5. The backend copies scenario metadata and issues into the `ExamSessions` table. The copied data preserves the exact exam version even if the original scenario changes later.
6. The backend creates a `META` record for the session and one `ISSUE#<issueId>` record per issue.
7. The first issue is visible immediately. Later issues receive calculated release times across the 180-second exam duration.

8. For delayed issues, the backend sends SQS messages with `DelaySeconds`.
9. When SQS delivers a message, the `ReleaseExamIssueFunction` marks that issue visible in DynamoDB.
10. The frontend polls `/exam/sessions/{sessionId}/pulse`. The pulse endpoint returns remaining time and visible issues.
11. The pulse endpoint also reveals due issues by comparing their `releaseAt` time with the current time. This makes issue release resilient even if an SQS delivery is delayed or a browser poll arrives first.
12. The representative submits responses to visible issues and marks completed issues done.

The backend rejects answers to hidden issues, rejects writes to sessions owned by another user, and rejects response changes after a session has ended.

7. AI Evaluation and Dashboard Flow

After the 180-second exam window ends, the representative can request an evaluation.

1. The frontend calls `POST /exam/sessions/{sessionId}/evaluation`.
2. The backend verifies that the requester owns the session and that the session has ended.
3. The backend reads all session issue records and representative responses from DynamoDB.
4. The backend reads the OpenAI API key from Secrets Manager.
5. The backend calls the OpenAI Responses API with a strict JSON schema.
6. OpenAI returns rubric scores, per-issue scores, notes, strengths, growth areas, and practice ideas.
7. The backend normalizes scores to a 0-100 scale, calculates the weighted score, writes an `EVALUATION` record, and marks the session ended.
8. The representative results page reads the persisted evaluation through `GET /exam/sessions/{sessionId}/evaluation`.

Managers use `/dashboard` to review performance. The dashboard Lambda scans exam sessions, user profiles, and scenarios, groups session records by `sessionId`, joins representative names, and calculates summary metrics:

- total attempts.
- active and completed attempts.
- evaluated attempts.
- average success score.
- pass rate using an 80-point pass score.
- representatives evaluated.
- sessions that still need evaluation.
- score bands and per-representative coaching focus.

For the current project scale this scan-based dashboard is acceptable. For larger production use, the next architecture step would add DynamoDB GSIs or precomputed analytics records to avoid full-table scans.

8. Deployment and Configuration

Infrastructure is managed with AWS SAM and CloudFormation from `backend/template.yaml`. Deployment outputs include the API base URL, health URL, DynamoDB table names, SQS queue URL, Cognito User Pool ID, and Cognito app client ID.

The backend targets `us-east-1` in the AWS Academy lab. Lambda functions use Node.js `nodejs24.x`, 128 MB memory by default, and short timeouts. AI-related functions use higher timeout and memory settings:

- issue generation: 25-second timeout and 256 MB.

- exam evaluation: 30-second timeout and 256 MB.
- dashboard aggregation: 15-second timeout and 256 MB.

The root project scripts provide build and verification commands:

- `npm run dev` starts the frontend.
- `npm run typecheck` verifies TypeScript.
- `npm run build` builds the frontend.
- `npm run smoke` checks the deployed health endpoint.
- `npm run sam:build` builds the AWS SAM backend.
- `npm run sam:deploy:guided` deploys the backend stack.

The frontend connects to the deployed API through `frontend/.env.local`:

```
VITE_API_BASE_URL=https://<api-id>.execute-api.us-east-1.amazonaws.com/dev
```

Secrets are not committed. AWS credentials stay in the ignored root `.env`, frontend local API configuration stays in `frontend/.env.local`, and OpenAI credentials stay in AWS Secrets Manager.

9. Reliability, Security, and Scaling Notes

The architecture is intentionally serverless. API Gateway, Lambda, DynamoDB pay-per-request tables, Cognito, Secrets Manager, and SQS reduce operational work and fit the project requirement to use AWS serverless services.

Reliability comes from several design choices:

- The frontend never talks directly to DynamoDB, SQS, Cognito admin APIs, or Secrets Manager.
- API Gateway validates Cognito tokens before protected Lambda invocation.
- Lambda handlers perform role checks against the `Users` table instead of trusting frontend routing.
- Exam sessions copy scenario issue data, so completed attempts remain stable even if managers edit the original scenario.
- SQS handles delayed issue release, while the pulse endpoint also reveals due issues as a fallback.
- AI outputs are constrained with strict JSON schemas and normalized before storage.
- OpenAI issue generation has a demo fallback so the lab flow can continue if the external provider is unavailable.

Security boundaries are clear:

- Cognito owns passwords and token refresh.
- DynamoDB stores only app profile and business data.
- Secrets Manager stores the OpenAI key.
- Representatives can access only their own exam sessions.
- Managers can administer users and content, but cannot demote or suspend themselves.
- Suspended users are blocked by backend authorization checks.

The main production hardening items are to replace wildcard CORS with the final frontend origin, add least-privilege IAM policies instead of the shared AWS Academy `LabRole`, add analytics indexes for dashboard scale, and place the frontend behind a managed static hosting layer such as Amazon S3 and CloudFront if the final deployment requires public web hosting.

10. Section 02 Diagram Alignment

The Section 02 architecture diagram should use official AWS icons and show these connections:

- User browser to React single-page application.

- React application to API Gateway REST API over HTTPS.
- API Gateway to Cognito authorizer for protected routes.
- API Gateway to Lambda functions for auth, content, exam, dashboard, and health routes.
- Auth Lambdas to Cognito User Pool and the `Users` DynamoDB table.
- Content Lambdas to `Users`, `Personas`, and `Scenarios`.
- Issue generation and evaluation Lambdas to Secrets Manager and the external OpenAI Responses API.
- Exam session Lambda to `Scenarios`, `ExamSessions`, and SQS.
- SQS to the issue-release Lambda.
- Issue-release Lambda back to `ExamSessions`.
- Dashboard Lambda to `ExamSessions`, `Users`, and `Scenarios`.

This set of boxes and arrows matches the implemented SAM template and source code, and it is the architecture that this document explains.